Aplicaciones web II

Analista de Sistemas y Desarrollo de Software

Integrantes:

- Nievas Rocío
- Chanquia Antonella
- Cordoba Milagros
- Casagrande Maria Trinidad

Índice

.....0
Proyecto Web – “SANTIAGO CEREALES SRL” 2
1. Introducción 2
2. Equipo de trabajo y roles.....3
3. Descripción del proyecto 3
4. Justificación 4
5. Objetivos.....4
6. Cronograma de trabajo 5
7. Herramienta de gestión del proyecto.....5
8. Herramienta de control de versiones.....6
9. Repositorio.....6
10. Herramienta de seguimiento 6
11. Conclusión 6
12. Avance TP2 — Back-end para servir el sitio web 7
13. Avance TP3 — Desarrollo del CRUD completo.....10

Proyecto Web – “SANTIAGO CEREALES SRL”

TP1 – Propuesta de Desarrollo del Proyecto

1. Introducción

El presente documento tiene como objetivo presentar la propuesta de desarrollo del proyecto que se llevará a cabo durante la materia Aplicaciones Web II.

Como base del trabajo se utilizará el proyecto **Santiago Cereales SRL**, desarrollado previamente en la materia Aplicaciones Web I. Durante esta nueva etapa se buscará evolucionar dicho proyecto incorporando funcionalidades del lado del servidor (Back-End), manteniendo el Front-End previamente desarrollado y ampliando sus capacidades.

El propósito es continuar el desarrollo de una aplicación web que pueda convertirse en una solución funcional para la empresa, permitiendo administrar la información de manera más eficiente y sentando las bases para una futura implementación real.

Visión del proyecto

El proyecto busca transformar el sitio web institucional desarrollado en Aplicaciones Web I en una aplicación web completa, incorporando progresivamente las tecnologías y conocimientos adquiridos durante la materia Aplicaciones Web II. Además del objetivo académico, se pretende desarrollar una solución que pueda ser presentada como propuesta real para la empresa Santiago Cereales SRL

---

2. Equipo de trabajo y roles

El proyecto será desarrollado por un equipo de cuatro integrantes. Cada miembro tendrá un rol principal para facilitar la organización del trabajo, aunque las responsabilidades podrán rotarse a lo largo del cuatrimestre con el objetivo de que todos los integrantes adquieran experiencia en las distintas áreas del desarrollo Back-End.

Integrante	Rol principal	Responsabilidades
Rocío Nievas	Líder del proyecto y Desarrollo Back-End	Coordinación del equipo, planificación, desarrollo de la lógica del servidor, integración con el Front-End y seguimiento de GitHub Projects.
Antonella Chanquia	Desarrollo Back-End	Implementación de funcionalidades, colaboración en el desarrollo del servidor, documentación técnica y control de versiones.
Milagros Córdoba	Base de Datos y API	Diseño y administración de la base de datos, creación de modelos, integración con la API y realización de pruebas de funcionamiento.
Maria Trinidad Casagrande	Testing, Documentación y Calidad	Pruebas del sistema, detección de errores, validación de funcionalidades, actualización de la documentación y colaboración en las entregas.

---

3. Descripción del proyecto

El proyecto consiste en continuar el desarrollo del sitio web **Santiago Cereales SRL**, realizado en la materia Aplicaciones Web I, incorporando nuevas funcionalidades correspondientes al desarrollo Back-End.

En la primera etapa se desarrolló un sitio web institucional que permite a los usuarios conocer la empresa, sus servicios, productos y medios de contacto.

Durante Aplicaciones Web II se buscará ampliar dicho proyecto incorporando nuevas funcionalidades que permitan gestionar la información del sistema de manera organizada, manteniendo el sitio desarrollado previamente como interfaz para los usuarios.

De esta forma, el proyecto evolucionará progresivamente durante la cursada hasta transformarse en una aplicación web más completa.

El desarrollo del Back-End se llevará a cabo utilizando Node.js y el framework Express, que permiten crear un servidor propio encargado de publicar el sitio y de exponer rutas (endpoints) propias. Estas rutas consultan los datos a través de una API externa (MockAPI), evitando que la información quede almacenada en archivos estáticos como ocurría en la etapa anterior. A lo largo de la cursada, este servidor irá incorporando progresivamente nuevas capacidades: en primer lugar la conexión con la API externa (TP2), luego operaciones de creación, lectura, actualización y eliminación de datos —CRUD— (TP3), y finalmente un sistema de autenticación y seguridad para proteger dichas operaciones (TP4).

---

4. Justificación

Actualmente el sitio web desarrollado presenta la información de forma estática, siendo necesario modificar el código fuente para realizar cualquier actualización.

La continuidad del proyecto permitirá incorporar herramientas que faciliten la administración de la información y mejoren la organización del sistema.

Además del objetivo académico, el proyecto busca generar una solución que pueda adaptarse a una necesidad real de la empresa Santiago Cereales SRL, constituyendo una base para una futura implementación.

---

5. Objetivos

Objetivo general

Continuar el desarrollo del proyecto Santiago Cereales SRL incorporando las funcionalidades correspondientes a la materia Aplicaciones Web II.

Objetivos específicos

- Continuar utilizando el Front-End desarrollado en Aplicaciones Web I.
- Incorporar progresivamente nuevas funcionalidades durante la cursada.
- Mantener una estructura ordenada del proyecto.
- Aplicar buenas prácticas de desarrollo y documentación.
- Gestionar correctamente las versiones del proyecto.
- Implementar un servidor propio utilizando Node.js y Express.
- Conectar el sistema a una API externa para el manejo de los datos, evitando la dependencia de archivos estáticos.
- Incorporar progresivamente operaciones CRUD y mecanismos de autenticación en las siguientes etapas del proyecto.

---

6. Cronograma de trabajo

Entrega	Actividad	Fecha límite
TP1	Propuesta de desarrollo	28/08/2026
TP2	Implementación del servidor para Front-End	18/09/2026
TP3	Desarrollo del CRUD	16/10/2026
TP4	Desarrollo del Login y seguridad del CRUD	06/11/2026

Las actividades se desarrollarán respetando el cronograma establecido por la cátedra.

---

7. Herramienta de gestión del proyecto

Para la organización del proyecto se utilizará **GitHub Projects**.

Esta herramienta permitirá planificar las tareas, asignar actividades, realizar el seguimiento del avance del proyecto y mantener un control organizado de cada una de las etapas de desarrollo.

Las tareas serán clasificadas en distintas columnas según su estado de avance, facilitando el trabajo colaborativo entre ambos integrantes.

---

8. Herramienta de control de versiones

Se utilizará **Git** como sistema de control de versiones y **GitHub** como plataforma para alojar el repositorio del proyecto.

El uso de Git permitirá registrar el historial de modificaciones realizadas durante el desarrollo, facilitando el trabajo colaborativo y el seguimiento de los avances mediante commits realizados dentro de las fechas establecidas por la cátedra.

---

9. Repositorio

El repositorio contendrá:

- Código fuente del proyecto.
- Documento PDF correspondiente al TP1.
- Archivos del proyecto.
- Historial de commits.
- Documentación necesaria para el desarrollo.

---

10. Herramienta de seguimiento

Se utilizará **GitHub Projects** para gestionar las tareas del proyecto.

El tablero permitirá visualizar el estado de cada actividad mediante columnas de trabajo como:

- Pendiente
- En desarrollo
- En revisión
- Finalizado

---

11. Conclusión

El presente proyecto representa la continuidad del trabajo realizado en Aplicaciones Web I, permitiendo evolucionar el sitio web desarrollado hacia una aplicación con mayores funcionalidades.

Durante el transcurso de la materia se incorporarán progresivamente nuevos conocimientos y herramientas propias del desarrollo Back-End, manteniendo una metodología de trabajo organizada mediante el uso de Git, GitHub y GitHub Projects.

El objetivo final es obtener una aplicación web funcional, escalable y bien documentada, que además de cumplir con los requisitos académicos, pueda convertirse en una propuesta de valor para una futura implementación en la empresa Santiago Cereales SRL.

.

---

12. Avance TP2 — Back-end para servir el sitio web

Este apartado documenta el desarrollo correspondiente al TP2, cuyo objetivo fue transformar el sitio estático desarrollado en Aplicaciones Web I en una aplicación servida por un back-end propio, que obtiene sus datos desde una API externa en lugar de un archivo JSON local.

Tecnologías utilizadas

- **Node.js:** entorno de ejecución de JavaScript del lado del servidor.
- **Express.js:** framework utilizado para crear el servidor y definir las rutas (endpoints).
- **ES Modules (.mjs, import/export):** sistema de módulos utilizado para organizar el código en archivos separados.
- **MockAPI:** plataforma utilizada para simular una API externa, donde se cargaron los datos de categorías y productos de la empresa.

Arquitectura implementada

El Front-End no se conecta directamente a MockAPI. Se implementó un servidor propio que actúa como intermediario entre el Front-End y la fuente de datos externa:

```
Front-End (HTML / CSS / JS)
        |
        v
Servidor propio (Node.js + Express)
        |
        v
MockAPI (datos externos)
```

- Sirve los archivos estáticos del Front-End desarrollado en AW1, mediante express.static.
- Expone rutas propias que consultan los datos en MockAPI y se los devuelven al Front-End ya procesados.

Middlewares implementados

Un middleware es una función que se ejecuta entre la llegada de una petición al servidor y la generación de la respuesta. Recibe los objetos req (petición) y res (respuesta), más una función next() que debe invocar para que la petición continúe su camino hacia la siguiente función o ruta. Todos los middlewares se registran mediante app.use(). En el proyecto se utilizan los siguientes:

- **express.static("."):** middleware incluido en Express que sirve los archivos estáticos del Front-End (HTML, CSS, JS, imágenes) directamente desde la raíz del proyecto.
- **productosRouter:** el conjunto de rutas propias (/categorias, /productos/:categoriaId, /productos) se organiza en un Router de Express y se registra como middleware en server.mjs.
- **express.json():** middleware incluido en Express que procesa el cuerpo (body) de las peticiones que llegan en formato JSON y lo deja disponible en req.body. Es necesario para poder leer los datos enviados por método POST.
- **logger (propio):** middleware simple, escrito para este proyecto, que registra en la consola del servidor el método HTTP y la ruta de cada petición recibida (por ejemplo, POST /productos), a modo de evidencia del funcionamiento del servidor.

```
function logger(req, res, next) {
    console.log(`${req.method} ${req.url}`);
    next();
}

app.use(logger);
app.use(express.json());
app.use(express.static("."));
app.use(productosRouter);
```

Modelo de datos

Se definieron dos recursos relacionados en MockAPI:

- **categorias:** las 5 categorías de productos de la empresa (Herbicidas, Fungicidas, Fertilizantes, Semillas, Insecticidas), cada una con nombre e imagen.

- **productos:** productos individuales, cada uno con nombre, marca, descripcion, imagen y categoriaId (referencia a la categoría a la que pertenece).

Estructura de carpetas del proyecto

```
/ (raíz del proyecto)
  server.mjs            -> configura y levanta el servidor
  /routes
    productos.mjs        -> define los endpoints propios
  /css, /js, /recursos  -> Front-End existente (sin cambios de
organización)
  index.html, empresa.html, contacto.html, serviciosyproductos.html
  package.json
```

Endpoints desarrollados

Método	Ruta	Descripción
GET	`(base-url)/categorias`	Obtiene todas las categorías de productos disponibles.
GET	`(base-url)/productos/:categoriaId`	Obtiene los productos pertenecientes a una categoría específica, identificada por su ID.
POST	`(base-url)/productos`	Crea un nuevo producto: recibe sus datos en el cuerpo de la petición (JSON) y los envía a MockAPI para almacenarlos.

Integración con el Front-End existente

Se modificó js/productos.js para que, en lugar de leer productos.json de forma local, consuma los endpoints propios del servidor. Al hacer click en una categoría, se solicitan sus productos (con marca y descripción) al segundo endpoint y se muestran dinámicamente en pantalla.

El endpoint POST /productos, en cambio, no se consume todavía desde una pantalla del Front-End: la carga de productos nuevos forma parte del CRUD que se desarrollará en el TP3. Por el momento se probó de forma manual con la extensión Thunder Client, enviando una petición POST con un producto en formato JSON y verificando en la consola del servidor y en MockAPI que se creaba correctamente.

Dificultades encontradas y resueltas

- Diferencia de tipos de datos entre el parámetro recibido en la URL (siempre texto) y el campo categoriaId almacenado como número en MockAPI, que impedía que el filtro

encontrara coincidencias. Se resolvió convirtiendo el valor con Number() antes de comparar.

- Error de referencia de archivos al ejecutar el servidor, por tener server.mjs ubicado en una carpeta distinta a la raíz del proyecto.

---

13. Avance TP3 — Desarrollo del CRUD completo

Este apartado documenta el desarrollo correspondiente al TP3, cuyo objetivo fue completar las operaciones CRUD (Alta, Baja, Modificación y Lectura) sobre los productos del catálogo, reemplazando la fuente de datos externa (MockAPI) por una base de datos propia, y documentando la API resultante.

Tecnologías utilizadas

- **PostgreSQL:** motor de base de datos relacional utilizado para reemplazar a MockAPI como fuente real de datos.
- **Docker:** utilizado para levantar un contenedor con PostgreSQL ya configurado, sin necesidad de instalar el motor de base de datos directamente en cada máquina.
- **pg (node-postgres):** cliente oficial de Node.js para conectarse a PostgreSQL y ejecutar consultas SQL desde el servidor.
- **dotenv:** utilizado para leer las credenciales de conexión a la base de datos desde un archivo .env, evitando escribirlas directamente en el código fuente.
- **Multer:** middleware de Express utilizado para procesar formularios con archivos, permitiendo subir imágenes de productos desde la computadora en lugar de escribir una URL a mano.
- **Swagger (swagger-jsdoc y swagger-ui-express):** utilizados para generar documentación interactiva de la API a partir de comentarios escritos sobre cada ruta.

Arquitectura implementada

El servidor se reorganizó siguiendo el patrón MVC (Modelo-Vista-Controlador), separando cada módulo del proyecto (categorías y productos) en tres archivos con responsabilidades distintas:

- **Rutas (*.rutas.mjs):** definen las URLs disponibles y qué función del controlador se ejecuta para cada una.
- **Controlador (*.controlador.mjs):** recibe la petición, coordina la respuesta y maneja los errores.

- **Modelo (*.modelo.mjs):** es el único archivo que se comunica con la base de datos.

Esta separación permitió que, al migrar de MockAPI a PostgreSQL, solo fuera necesario modificar los archivos modelo, sin tocar las rutas ni los controladores.

Además, todas las rutas propias del proyecto pasaron a usar el prefijo /api/v1/ (por ejemplo, /api/v1/productos), y el Front-End (HTML, CSS, JS e imágenes) se separó en su propia carpeta web/, independizándolo de los archivos del servidor.

Base de datos

Se reemplazó MockAPI por una base de datos PostgreSQL real, levantada mediante un contenedor de Docker. Se definieron dos tablas relacionadas:

- **categorias:** id (clave primaria autoincremental), nombre, imagen.
- **productos:** id (clave primaria autoincremental), nombre, marca, descripcion, imagen, categoria_id (clave foránea que referencia a categorias.id).

A diferencia de MockAPI, donde la relación entre un producto y su categoría era un simple número sin validación, en PostgreSQL esa relación está garantizada por la base de datos mediante una restricción REFERENCES, que impide crear un producto con una categoría inexistente.

Las credenciales de conexión (usuario, contraseña, nombre de la base) se guardan en un archivo .env, que no se sube al repositorio por motivos de seguridad; en su lugar se incluye un .env.example con la misma estructura, sin los valores reales, para que cualquiera que clone el proyecto sepa qué variables debe completar. Por tratarse de una base de datos local (en Docker), cada integrante del equipo debe levantar su propio contenedor en su computadora para poder trabajar.

Estructura de carpetas actualizada

```
Santiago Cereales SRL/
  server.mjs
  swagger.mjs
  .env.example
  /db
    Dockerfile
    santiago_cereales.sql
    conexion.mjs
  /modulos
    /categorias  -> categorias.rutas.mjs, categorias.controlador.mjs,
categorias.modelo.mjs
    /productos   -> productos.rutas.mjs, productos.controlador.mjs,
```

```
productos.modelo.mjs
  /web
    /css, /js, /recursos, /paginas, index.html
  package.json
  pnpm-lock.yaml
```

Endpoints desarrollados

Método	Ruta	Descripción
GET	`/api/v1/categorias`	Obtiene todas las categorías de productos disponibles.
GET	`/api/v1/productos`	Obtiene todos los productos cargados.
GET	`/api/v1/productos/categoria/:categoriaId`	Obtiene los productos pertenecientes a una categoría específica.
GET	`/api/v1/productos/:id`	Obtiene un producto puntual a partir de su id.
POST	`/api/v1/productos`	Da de alta un producto nuevo (incluye la imagen, subida como archivo).
PUT	`/api/v1/productos/:id`	Modifica los datos de un producto existente.
DELETE	`/api/v1/productos/:id`	Elimina un producto del catálogo.

Subida de imágenes con Multer

En el TP2, el campo “imagen” de un producto se cargaba escribiendo una URL a mano. En esta etapa se integró Multer, un middleware de Express que permite procesar formularios de tipo multipart/form-data (los que incluyen archivos). Ahora, desde la pantalla de Alta/Modificación de productos (ABM), se puede elegir una imagen directamente desde la computadora: el archivo se guarda en el servidor, conservando su extensión original, y en la base de datos solo se almacena la ruta hacia ese archivo, no la imagen completa. Si al modificar un producto no se selecciona una imagen nueva, el sistema conserva automáticamente la que ya tenía.

CORS

La consigna indica: “Exponer los endpoints de lectura. Habilite CORS para ambos si es necesario.” Se evaluó esta necesidad y se determinó que no es necesario habilitar CORS, ya que el Front-End y la API se sirven desde el mismo servidor y el mismo origen

(http://localhost:3000). CORS es un mecanismo de seguridad de los navegadores que solo entra en juego cuando una página realiza peticiones hacia un origen (dominio/puerto) distinto al propio; al no existir ese escenario en este proyecto, no se incorporó el paquete cors.

Documentación de la API con Swagger

Se documentó la totalidad de los endpoints utilizando Swagger, a través de los paquetes swagger-jsdoc (que genera la documentación a partir de comentarios escritos sobre cada ruta) y swagger-ui-express (que expone una página interactiva para consultarla). La documentación queda disponible en http://localhost:3000/api/docs, donde se puede ver el detalle de cada endpoint (parámetros, formato esperado, respuestas) y probarlo en vivo desde la misma página, incluyendo el envío de imágenes en los endpoints de Alta y Modificación.

Integración con el Front-End

A diferencia del TP2, donde el endpoint POST solo se había probado con Thunder Client, en esta etapa se desarrolló por completo la pantalla de Gestión de Productos (ABM), que permite:

- **Dar de alta** un producto nuevo, incluyendo su imagen.
- **Modificar** los datos de un producto existente, conservando su imagen si no se carga una nueva.
- **Eliminar** un producto, con confirmación previa.
- **Listar** todos los productos cargados en una tabla, con su categoría correspondiente.

Además, se corrigió la forma en que se muestran las imágenes de los productos en la pantalla de “Servicios y Productos” (antes se recortaban al no respetar su proporción original), y se mejoró el estilo visual del formulario para que todos sus campos (incluido el selector de categoría) tengan una apariencia consistente.

Dificultades encontradas y resueltas

- Diferencia de tipos de datos entre el id de categoría recibido como texto (desde el HTML) y el id numérico devuelto por PostgreSQL, que impedía encontrar la categoría correspondiente al mostrar el detalle de productos. Se resolvió convirtiendo el valor con Number() antes de comparar.
- Un archivo del Front-End (abmProductos.js) se perdió durante la reorganización del proyecto y tuvo que reconstruirse.
- Las imágenes de los productos se veían recortadas en las tarjetas, por el uso de object-fit: cover; se solucionó cambiando a object-fit: contain.

- Al migrar a PostgreSQL, cada integrante del equipo necesita levantar su propio contenedor de Docker y su propio archivo .env en su computadora, ya que la base de datos no es compartida entre máquinas.